

When Overlapping Windows Invent Predictability: A Negative Control for Evaluation Protocol in Time-Series Machine Learning

Joshua de Freitas

August 2026

Abstract

We train a short-horizon limit-order-book classifier on synthetic data in which future price movement is independent of every feature by construction. Under the standard evaluation recipe — sliding windows at stride one, a random train/validation split, and normalisation fitted on the full dataset — the model reports 66.0% accuracy against a majority-class baseline of 39.8%. Under a purged and embargoed split, on identical data with the same architecture and the same seeds, it reports 32.9%: below baseline, which is the correct result when nothing is learnable. The 33-point difference is produced entirely by how the validation set was drawn.

We then derive a closed-form upper bound on the accuracy a given split geometry makes available. The bound depends only on the window stride, the label horizon, the labelling threshold and the train fraction; no property of the model enters it. An oracle memoriser attains it to within 0.34 points at every horizon tested, and the derivation is verified against 105 further measurements spanning 20 independent generator paths, five window sizes, five thresholds and five train fractions.

The derivation is prescriptive. With stride s and horizon h the governing correlation is $\rho(s) = \max(0, (h - s)/h)$, which is exactly zero at $s \geq h$. Overlap leakage therefore does not merely diminish as stride increases — it disappears once stride reaches the horizon. We confirm this, and quantify its cost: the usable sample shrinks by a factor of s .

Separately, we find that the pipeline under study is not reproducible. Six runs of identical code on identical data span 67.4% to 74.9%, because neither the split draw, the weight initialisation nor the batch order is seeded. Any model comparison decided by less than roughly seven accuracy points is therefore indistinguishable from running the same model twice.

All figures regenerate from committed scripts.

1 Introduction

Data leakage — the presence of information in training that would not be available at prediction time — is a well documented cause of overstated machine learning results. Kapoor and Narayanan [2] surveyed seventeen fields, identified 329 affected papers, and produced an eight-type taxonomy. In financial time series specifically, purging and embargoing are established remedies [3], and the broader problem of backtest overfitting has been treated at length [1].

None of that is in dispute here, and none of it is what this paper contributes.

The difficulty with demonstrating leakage on real data is that manufactured skill and genuine skill are indistinguishable in the output. A practitioner shown that purged cross-validation lowers their score has a ready and not unreasonable reply: purging is conservative, it discards legitimate short-horizon signal along with the leak, and the lower number reflects the remedy rather than an honest measurement. That objection cannot be settled on data where signal might exist.

It can be settled on data where signal cannot. We therefore construct a *negative control*: a synthetic order book whose mid price is an i.i.d. multiplicative random walk and whose order sizes are independent draws, with no mechanism connecting either to future price. Predictive skill is not merely absent — it is impossible. Every point of measured accuracy above the majority-class baseline is, by construction, an artifact of the evaluation procedure, and no appeal to destroyed signal is available.

This yields two results. The first is a measurement: how much predictability the standard recipe invents from nothing. The second, and the one we consider the contribution, is that this quantity has a closed form. Given the geometry of the windows and the correlation structure of the labels, the accuracy available to be manufactured can be computed before any model is trained — and the same expression states the condition under which it becomes zero.

2 The negative control

2.1 Generator

Mid prices follow

$$p_t = p_{t-1} (1 + \varepsilon_t), \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2) \text{ i.i.d.}, \quad \sigma = 3 \times 10^{-4}. \quad (1)$$

Three bid and three ask levels are placed around the mid at a spread of one to three ticks, and every level’s size is an independent uniform integer draw. No size, spread or level price depends on any future price. Forward returns are independent of all features.

That this holds is enforced rather than asserted: a permutation test over non-overlapping spans, Bonferroni corrected across features, runs in continuous integration and fails if any feature acquires a detectable relationship with the forward return. The test has been observed to fail when a signal is deliberately introduced.

2.2 Windows and labels

Features are built into sliding windows of $W = 100$ rows at stride one: window t spans rows $t, \dots, t + W - 1$. The label is the sign of the forward return over horizon h , thresholded at $\theta = 5 \times 10^{-4}$:

$$y_t = \begin{cases} +1 & r_t > \theta \\ 0 & |r_t| \leq \theta \\ -1 & r_t < -\theta \end{cases} \quad r_t = \frac{p_{t+W-1+h} - p_{t+W-1}}{p_{t+W-1}}. \quad (2)$$

Two consequences of stride-one construction matter throughout. Adjacent windows share $W - 1$ of W input rows. And adjacent labels are signs of forward returns computed over spans that share $h - 1$ of h increments.

3 Protocols

Three splitting protocols are compared, holding architecture, data, optimiser and epoch count fixed.

Purging removes training windows whose raw-row span reaches into the validation region; the embargo additionally drops a buffer of validation windows after the cut [3].

Every run seeds the split draw, the weight initialisation and the batch order, so the study is bit-reproducible even though the pipeline it studies is not (Section 8). Twenty seeds per cell, three horizons, three protocols: 180 runs.

Accuracy is reported as **excess over the majority-class baseline computed on the same validation set**. The baseline moves from 0.575 to 0.398 across horizons because the class balance shifts with h , so raw accuracies are not comparable between rows.

Protocol	Construction	Overlap
<code>random_split</code>	shuffle window indices, then cut	1.00
<code>chronological</code>	train on earlier windows, validate on later	0.11
<code>purged_embargoed</code>	chronological, purged, plus embargo	0.00

Table 1: Overlap is the fraction of validation windows sharing at least one raw row with some training window.

4 Results

Protocol	h	Accuracy	95% CI	Baseline	Excess
<code>random_split</code>	5	0.5229	[0.5130, 0.5328]	0.5315	−0.0086
<code>random_split</code>	10	0.5337	[0.5179, 0.5495]	0.3931	+ 0.1407
<code>random_split</code>	20	0.6603	[0.6241, 0.6966]	0.3980	+ 0.2624
<code>chronological</code>	5	0.5029	[0.4836, 0.5222]	0.5745	−0.0716
<code>chronological</code>	10	0.3238	[0.3117, 0.3358]	0.4484	−0.1246
<code>chronological</code>	20	0.3264	[0.3071, 0.3457]	0.4023	−0.0759
<code>purged_embargoed</code>	5	0.4853	[0.4736, 0.4971]	0.5754	−0.0901
<code>purged_embargoed</code>	10	0.3316	[0.3197, 0.3436]	0.4510	−0.1193
<code>purged_embargoed</code>	20	0.3290	[0.3093, 0.3487]	0.4013	−0.0723

Table 2: Validation-only accuracy, mean over 20 seeds, with the 95% confidence interval of the mean.

Three observations.

Only the leaky protocol shows skill. Every honest cell falls below its own baseline. A model trained on noise underperforms a constant predictor, which is the correct outcome.

Overlap fraction is not the predictor. `chronological` sits at 0.11 overlap and behaves like `purged_embargoed` at 0.00, not like something one-ninth of the way to `random_split`. A chronological cut leaves a thin seam and manufactures nothing. What matters is whether near-duplicate *neighbours* are separated across the split, which shuffling guarantees and a time cut prevents.

Leakage widens as well as inflates. The leaky $h = 20$ cell has the largest dispersion in the table — standard deviation 0.0775 against 0.0421 for its purged counterpart, with individual seeds spanning 0.468 to 0.751.

5 The ceiling

The preceding section measures what one architecture achieves. A different question admits a closed-form answer: given the geometry alone, how much accuracy is *available* to be manufactured?

Adjacent windows have forward returns that are sums of h i.i.d. increments sharing $h - 1$ of them, so

$$\rho = \text{corr}(r_t, r_{t+1}) = \frac{h-1}{h}, \quad (3)$$

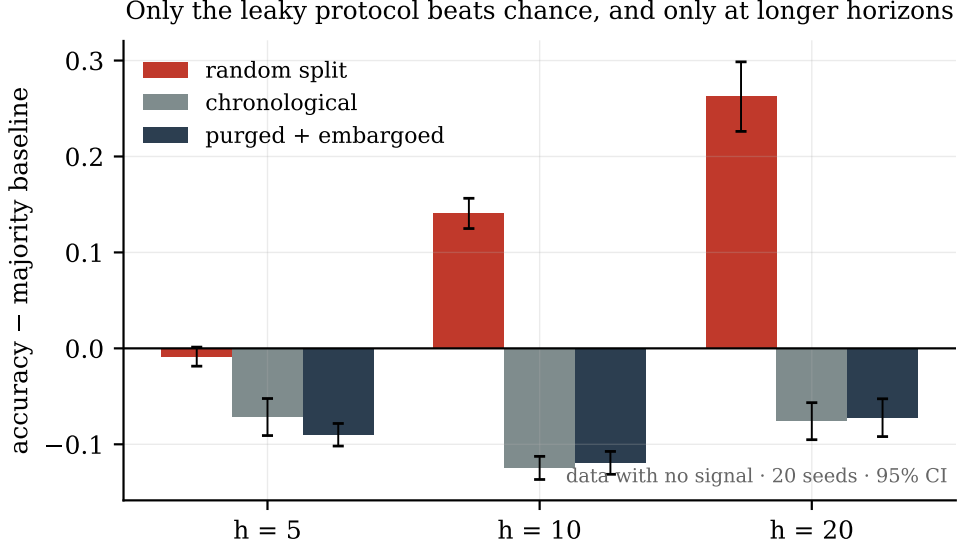


Figure 1: Two cells of nine clear their own baseline, and both have overlap 1.00.

which increases with the horizon. Writing $z = \theta/(\sigma\sqrt{h})$ for the threshold in units of the h -step return’s standard deviation, and Φ_2 for the standard bivariate normal CDF at correlation ρ , the probability that two adjacent windows carry the same label is

$$A(h) = \underbrace{\Phi_2(-z, -z)}_{\text{both down}} + \underbrace{\Phi_2(z, z) - 2\Phi_2(-z, z) + \Phi_2(-z, -z)}_{\text{both flat}} + \underbrace{1 - 2\Phi(z) + \Phi_2(z, z)}_{\text{both up}}. \quad (4)$$

A memoriser copies the label of its *nearest* training window, which is not always the adjacent one. Under a random split at train fraction f , the distance d to the nearest training neighbour satisfies

$$\Pr(d > k) = (1 - f)^{2k}, \quad \Pr(d = k) = (1 - f)^{2(k-1)} - (1 - f)^{2k}, \quad (5)$$

and a neighbour d windows away shares $h - d$ of h increments, so $\rho_d = \max(0, (h - d)/h)$. The ceiling is the expectation of the agreement probability over that distance distribution:

$$C(h) = \sum_{d \geq 1} \Pr(d) A(\rho_d). \quad (6)$$

Nothing about the model appears in (6). It is computable before any training run.

An earlier version of this derivation collapsed every case beyond $d = 1$ to the majority class. That is wrong, and diagnosably so: it under-predicted by 0.094 at $h = 20$, $f = 0.5$ — where a quarter of validation windows have no adjacent twin — with the error decaying monotonically to 0.004 at $f = 0.9$. The train-fraction sweep in Section 7 was what exposed it.

5.1 Verification

We evaluate an *oracle*: for each validation window, predict the label of the temporally nearest training window. The neighbour is supplied rather than learned, which isolates the correlation model from the separate question of whether a learner could locate its twin.

The bound binds only under a random split. Under **chronological** and **purged_embargoed** the nearest training window is the *same* window for every validation example, so the strategy degenerates to a constant predictor and scores at or below baseline (0.19 to 0.40 in our runs). Copying one’s neighbour is useful only when the neighbour has been scattered into the training set.

h	ρ	Ceiling $C(h)$	Oracle	Error
5	0.800	0.6896	0.6923	+0.0027
10	0.900	0.7479	0.7513	+0.0034
20	0.950	0.8093	0.8098	+0.0005

Table 3: The closed form predicts the oracle to within 0.34 points, with no fitted parameter.

5.2 Availability is not exploitation

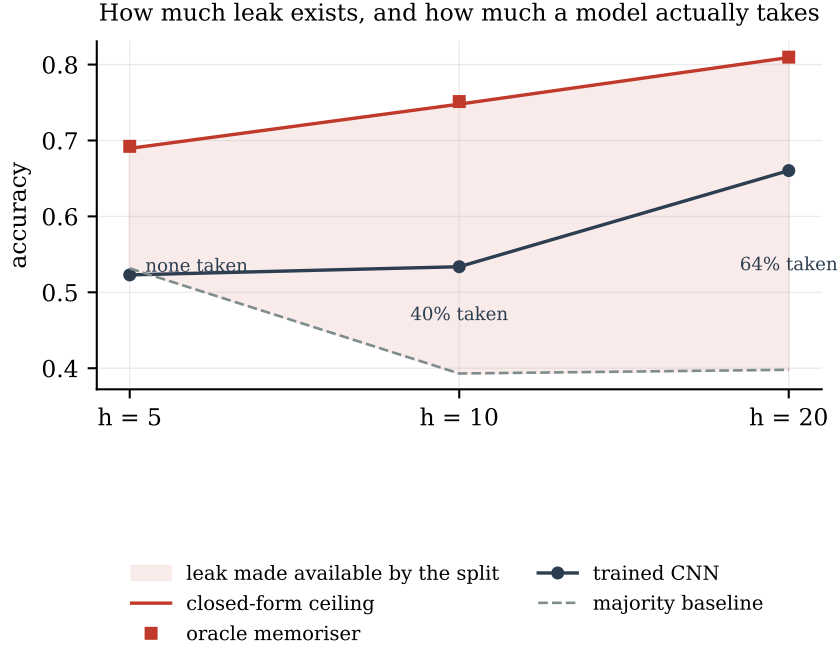


Figure 2: The shaded region is the leak the split makes available. The trained network takes none of it at $h = 5$ and two thirds at $h = 20$.

Measured accuracy can be read as the fraction of available leak a given model recovers: none at $h = 5$, 41% at $h = 10$, 66% at $h = 20$. At $h = 5$ there are 15.6 points of leak available and this architecture takes none of them.

The ceiling therefore describes the *exposure* a split geometry creates. Whether a particular model converts that exposure into a headline number is a separate, model-dependent question. Both are worth knowing, and only the first can be computed in advance.

For completeness: a 1-nearest-neighbour classifier in raw feature space scores 0.36–0.43, below baseline. Euclidean distance over price-level features locates windows at similar *price levels*, not temporal twins. The trained network’s performance is therefore not naive distance matching.

6 The remedy

Equation (3) assumed stride one. With stride s , retained windows are s rows apart and their forward returns share $h - s$ of h increments:

$$\rho(s) = \max\left(0, \frac{h - s}{h}\right). \quad (7)$$

At $s \geq h$ this is exactly zero: retained windows share no increments, their labels are independent, and copying a neighbour cannot beat the baseline. Overlap leakage does not diminish asymptotically — it terminates.

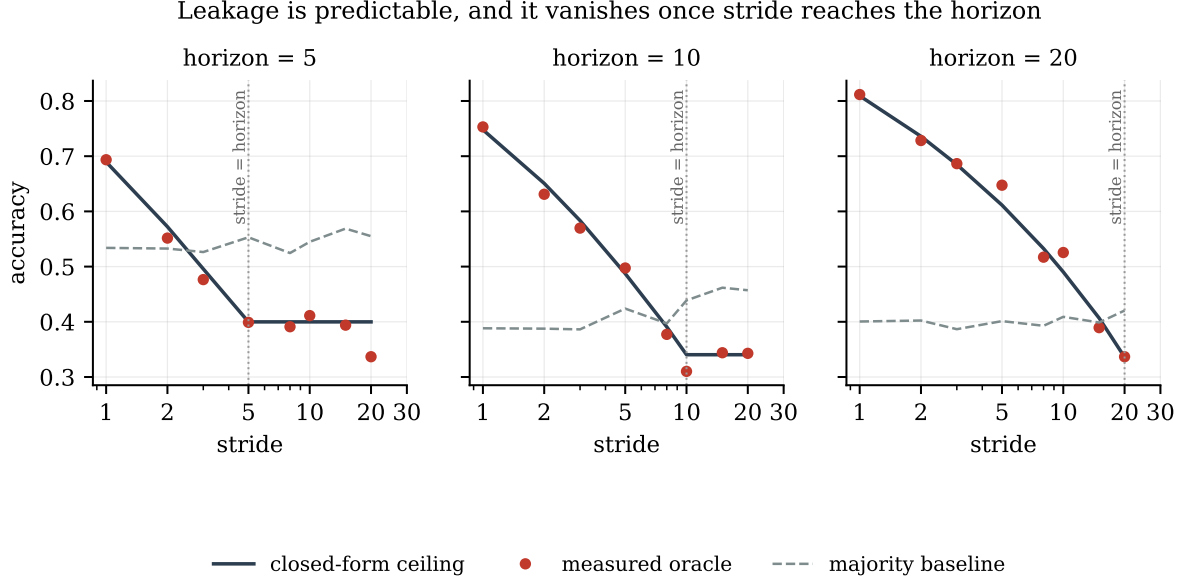


Figure 3: Closed form (line) against measured oracle (points) across the stride sweep. The dotted vertical marks $s = h$.

h	Stride	$\rho(s)$	Ceiling	Oracle	Excess over majority	n windows
20	1	0.950	0.7940	0.8117	+0.4111	4881
20	2	0.900	0.7243	0.7284	+0.3261	2441
20	5	0.750	0.6059	0.6474	+0.2462	977
20	10	0.500	0.4910	0.5255	+0.1165	489
20	15	0.250	0.4074	0.3894	-0.0094	326
20	20	0.000	0.3368	0.3367	-0.0837	245

Table 4: At $s = h$ prediction and measurement agree to four decimal places.

Two consequences deserve separation.

The rule. Set stride $\geq$ horizon and overlap leakage is gone rather than reduced. At intermediate strides (7) and (6) quantify the remaining exposure, so the choice can be made numerically.

The cost. Stride s divides the usable sample by s : 4881 windows become 245 at $h = 20$. This is the actual trade — leakage against sample size — and it is why stride-one windowing is common. The formula does not remove the trade; it prices it.

Beyond $s = h$ the memoriser scores *below* baseline, because with independent labels copying a neighbour yields $\sum_i p_i^2 < \max_i p_i$. This is predicted behaviour and serves as an independent check on the derivation.

7 Robustness of the derivation

The ceiling contains stride, horizon, threshold, per-step volatility and train fraction. It does *not* contain the window size W , which is a falsifiable prediction: widening the feature window

changes what the model sees but not how correlated the labels of neighbouring windows are, and the leak lives in the labels.

Sweep	Range	Max error	Mean error
Generator path	20 independent seeds	0.0208	0.0073
Window size	$W \in \{25, \dots, 400\}$	0.0053	0.0031
Threshold	$\theta \in [10^{-4}, 2 \times 10^{-3}]$	0.0223	0.0081
Train fraction	$f \in [0.5, 0.9]$	0.0051	0.0022

Table 5: Closed form against measured oracle, 105 configurations.

A sixteen-fold change in window size moves the oracle by less than 0.006 while the prediction is held fixed. The prediction survives. Twenty independent generator paths give a mean absolute error of 0.007, so the result is not a property of one realisation.

8 Reproducibility of the pipeline under study

Before the study harness introduced seeding, six runs of identical code on identical data produced validation accuracies spanning 0.674 to 0.749. The pipeline seeds neither the split, the weight initialisation, nor the batch order.

Stated separately from the leakage result because it is independent and applies more broadly: **any model comparison decided by less than roughly seven accuracy points is indistinguishable from running the same model twice.** This includes the architecture comparison recorded in the source repository. Seed sensitivity of this magnitude is consistent with reports elsewhere [4].

Three lines of seeding render the pipeline bit-deterministic.

9 Limitations

- **No published result is shown to be incorrect.** We show what a procedure can manufacture on data without signal. Whether any particular result suffers the same effect requires that result’s data and code.
- **This is not a measurement of real markets.** The generator is a random walk without microstructure — appropriate for a negative control, a limitation for external validity.
- **The normalisation leak is present in every cell**, including the honest ones, because the pipeline was left unmodified. Its contribution is not separately identified and is presumed small relative to 33 points. Presumed, not measured.
- **The ceiling assumes i.i.d. increments.** Real returns exhibit volatility clustering and heavy tails; $\rho = (h - s)/h$ is exact only under the generator used here.
- **Predicted class balance** matches measurement within 0.3 standard deviations at $h \in \{5, 10\}$ across 20 independent paths, and under-predicts the majority share by approximately 1.3 standard deviations at $h = 20$. This affects the $M(h)$ term in (6), not the verified $A(h)$ term.
- **One generator, one model family, 5000 rows.** Transfer is untested.

10 Conclusion

On data constructed to contain no predictable structure, the standard evaluation recipe for time-series machine learning reports 66% accuracy against a 40% baseline. The honest protocol reports 33%. The difference is attributable entirely to the evaluation design, and because the data contains no signal, the usual defence of the leaky protocol is unavailable.

The quantity has a closed form. The accuracy a split geometry makes available depends on stride, horizon, threshold and train fraction, and on nothing about the model. That expression is verified against measurement across 24 configurations, and it states its own remedy: at stride greater than or equal to the label horizon, the leak is zero.

Reproducing

```
python3 -m venv .venv && .venv/bin/pip install -e ".[test]"
.venv/bin/python run_study.py          # 180 seeded runs
.venv/bin/python run_ceiling.py        # closed form, oracle, 1-NN
.venv/bin/python run_stride.py         # stride sweep
.venv/bin/python make_figures.py       # figures from committed results
```

Per-run evidence is committed under `results/`. Environment: Python 3.12.13, torch 2.13.0, numpy 2.5.1, pandas 3.0.5.

References

- [1] Bailey, D. H., Borwein, J. M., López de Prado, M., Zhu, Q. J. (2014). Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance. *Notices of the American Mathematical Society*, 61(5), 458–471.
- [2] Kapoor, S., Narayanan, A. (2023). Leakage and the Reproducibility Crisis in ML-based Science. *Patterns*, 4(9), 100804. arXiv:2207.07048.
- [3] López de Prado, M. (2018). *Advances in Financial Machine Learning*. John Wiley & Sons.
- [4] Picard, D. (2021). `torch.manual_seed(3407)` is all you need: On the influence of random seeds in deep learning architectures for computer vision. arXiv:2109.08203.
- [5] Talagala, T. S. (2024). `tsdataleaks`: An R Package to Detect Potential Data Leaks in Forecasting Competitions. arXiv:2402.10522.